Object-Oriented Programming in Java

Java Exercises

Solutions

Solution code for all 28 exercises, compiled and run on OpenJDK 25 (tests with JUnit 5); every output matches the sample run in the exercise sheet. At the end: the lines of the slides that do not compile, or are wrong, in Java 25.

Contents

About these solutions **2**

Part 1 – Java environment **2**
1.1 Hello, command line . . . 2
1.2 Fix the conventions . . . 3
1.3 Jar file and magic number . . . 3

Part 2 – Java basics **3**
2.1 Calendar utilities . . . 3
2.2 Counter and Car . . . 5
2.3 Encapsulated bank account . . . 6
2.4 A shop in two packages . . . 7
2.5 Number tools . . . 9
2.6 Array tools . . . 10
2.7 static, final, factory methods, singleton . . . 11
2.8 Cards: enum and record . . . 12
2.9 Nested classes . . . 13

Part 3 – Characters and strings **14**
3.1 Text utilities . . . 14
3.2 A formatted receipt . . . 16
3.3 StringBuilder, speed and the string pool . . . 17
3.4 Encoding tools . . . 19

Part 4 – JUnit 5 **20**
4.1 Test the slides' Stack . . . 20
4.2 Test the bank account, then a suite . . . 21
4.3 Bug reproduction and TDD . . . 24

Part 5 – Inheritance, interfaces, lambdas **26**
5.1 Employees, managers, CEO . . . 26
5.2 equals, hashCode, toString, clone . . . 28
5.3 Abstract classes and Template Method . . . 29
5.4 Interfaces: complex numbers, default methods . . . 31
5.5 Composite expression tree . . . 32
5.6 Comparable, Comparator, Strategy . . . 33
5.7 Iterable and Iterator . . . 34
5.8 Functional interfaces, lambdas, method references . . . 35
5.9 Observer by composition . . . 36

Slide errata **38**

About these solutions

Every solution below was compiled and run on OpenJDK 25 (tests with JUnit 5), and its output matches the sample run in the exercise. The `main` methods are in the exercises, so mostly only the classes you write are shown. A code block often holds several files one after the other: put each public class in its own `.java` file. Obvious imports are left out: JUnit (`import static org.junit.jupiter.api.Assertions.*;` and `import org.junit.jupiter.api.*;`), `java.util.*`, `java.util.function.*`, `java.beans.*`, and your own earlier packages (e.g. import ex2_3.BankAccount; in 4.2). The full runnable project is in `java-exercises.zip` (one package per exercise under `src/`, tests under `test/`, `run-all.sh` builds and runs everything).

Part 1 – Java environment

1.1 Hello, command line

```
public class Greeter {
    public static void main(String[] args) {
        if (args.length == 0) {
            String name = IO.readln("Name? ");          // console input (Java 25 IO class)
            if (name == null || name.trim().isEmpty()) {
                name = "world";
            }
            IO.println("Hello, " + name + "!");
        }
        for (String name : args) {
            IO.println("Hello, " + name + "!");
        }
    }
}
```

```
javac Greeter.java        # creates Greeter.class (bytecode)
java Greeter Ada Linus    # class name, without .class
```

Java 25 form – the parameter stays because the program uses the arguments (otherwise `void main()`):

```
class GreeterCompact {
    void main(String[] args) {
        if (args.length == 0) {
            String name = IO.readln("Name? ");
            if (name == null || name.trim().isEmpty()) {
                name = "world";
            }
            IO.println("Hello, " + name + "!");
        }
        for (String name : args) {
            IO.println("Hello, " + name + "!");
        }
    }
}
```

1.2 Fix the conventions

Saved as `BankAccount.java` (a public class lives in a file with exactly its name):

```java
public class BankAccount {
    static final double INTEREST_RATE = 0.02;   // constants: UPPER_CASE
    private double balance;                     // attributes, methods, parameters: lowerCamelCase
    public void depositMoney(double amount) { balance += amount; }
    public double getBalance() { return balance; }
}
```

1.3 Jar file and magic number

```
javac Greeter.java
printf 'Main-Class: Greeter\n' > manifest.txt     # the manifest must end with a newline
jar cvfm greeter.jar manifest.txt Greeter.class
jar tf greeter.jar                                 # META-INF/  META-INF/MANIFEST.MF  Greeter.class
java -jar greeter.jar Ada                          # Hello, Ada!
java -cp greeter.jar Greeter Ada                   # Hello, Ada!  (no manifest needed)
```

Without the final newline the `Main-Class` line is ignored and `java -jar` answers `no main manifest attribute`. Shortcut: `jar cvfe greeter.jar Greeter Greeter.class` sets the entry point without a manifest file.

```java
import java.nio.file.Files;
import java.nio.file.Path;

public class MagicNumber {
    public static void main(String[] args) throws Exception {
        byte[] data = Files.readAllBytes(Path.of(args[0]));
        String hex = "";
        for (int i = 0; i < 4 && i < data.length; i++) {
            // bytes are signed (-128..127): & 0xFF turns e.g. (byte)0xCA = -54 into 202
            hex += String.format("%02X", data[i] & 0xFF);
        }
        IO.println(args[0] + " starts with " + hex);
        if (hex.equals("CAFEBABE")) {
            IO.println("It is a Java class file");
        } else {
            IO.println("It is not a Java class file");
        }
    }
}
```

Part 2 – Java basics

2.1 Calendar utilities

```java
public class CalendarUtils {

    static final String[] MONTHS = {
        "january", "february", "march", "april", "may", "june",
        "july", "august", "september", "october", "november", "december"
    };
```

```
    public static String season(String month) {
        String result;
        switch (month.toLowerCase()) {
            case "december":
            case "january":
            case "february":
                result = "winter";
                break;
            case "march":
            case "april":
            case "may":
                result = "spring";
                break;
            case "june":
            case "july":
            case "august":
                result = "summer";
                break;
            case "september":
            case "october":
            case "november":
                result = "autumn";
                break;
            default:
                result = "unknown";
        }
        return result;
    }

    public static boolean isLeap(int year) {
        return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    }

    public static int daysInMonth(String month, int year) {
        switch (month.toLowerCase()) {
            case "february":
                return isLeap(year) ? 29 : 28;
            case "april":
            case "june":
            case "september":
            case "november":
                return 30;
            case "january":
            case "march":
            case "may":
            case "july":
            case "august":
            case "october":
            case "december":
                return 31;
            default:
                return -1;
        }
    }

    public static int daysInYear(int year) {
        int total = 0;
        for (String m : MONTHS) {
            total += daysInMonth(m, year);
        }
        return total;
    }
```

```
    public static int nextLeapYear(int year) {
        int y = year + 1;
        while (!isLeap(y)) {
            y++;
        }
        return y;
    }

    public static void main(String[] args) {
        int[] years = {2023, 2024, 1900, 2000};
        if (args.length > 0) {
            years = new int[args.length];
            for (int i = 0; i < args.length; i++) {
                years[i] = Integer.parseInt(args[i]);
            }
        }
        for (int y : years) {
            String line = y + ": " + daysInYear(y) + " days";
            if (isLeap(y)) {
                line += " (leap)";
            }
            IO.println(line);
        }
        IO.println("Next leap year after 2025: " + nextLeapYear(2025));
        IO.println("JULY -> " + season("JULY") + ", Smarch -> " + season("Smarch"));
    }
}
```

2.2 Counter and Car

```
public class Counter {
    private int value;

    public Counter() { this(0); }                // delegates to Counter(int start)
    public Counter(int start) { value = start; }

    public Counter reset() { value = 0; return this; }
    public Counter increment(int by) { this.value += by; return this; }
    public Counter decrement(int by) { this.value -= by; return this; }
    public Counter print() { System.out.println(value); return this; }
    public int getValue() { return value; }
}

public class Car {
    private static int countBuiltCars = 0;

    private String brand;
    private String color;
    private boolean turnedOn;

    public Car() {
        this("Ferrari", "red");          // this(...) calls another constructor of Car
    }

    public Car(String brand) {
        this(brand, "white");
    }

    public Car(String brand, String color) {
        this.brand = brand;
```

```
        this.color = color;
        countBuiltCars++;                // every constructor ends up here
    }

    public Car(Car other) {              // copy constructor: a NEW, independent object
        this(other.brand, other.color);
        this.turnedOn = other.turnedOn;
    }

    public void turnOn() { turnedOn = true; }
    public void paint(String color) { this.color = color; }
    public void paint() { paint("white"); }      // overload: same name, other parameter list
    public boolean isOn() { return turnedOn; }
    public String getBrand() { return brand; }
    public String getColor() { return color; }

    public boolean sameAs(Car other) {   // contents, not identity
        return brand.equals(other.brand) && color.equals(other.color);
    }

    public static int getCountBuiltCars() { return countBuiltCars; }

    @Override
    public String toString() {
        return brand + " " + color + (turnedOn ? " (on)" : " (off)");
    }
}
```

`b = a` copies the reference (one object, two names), so painting `b` repaints `a`; `new Car(a)` makes a second object. The count is 5: `a`, `c`, the yellow Ferrari passed to `sameAs`, the Fiat and the Lancia.

2.3 Encapsulated bank account

```
public class BankAccount {
    public static final double MAX_WITHDRAWAL = 1000.0;
    private static int created = 0;

    private final int number;
    private final String owner;
    private double balance;

    public BankAccount(String owner) {
        this(owner, 0.0);
    }

    public BankAccount(String owner, double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Negative initial balance: " + initialBalance);
        }
        created++;                   // after the check: a rejected account uses no number
        this.number = created;
        this.owner = owner;
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid amount: " + amount);
        }
        balance += amount;
```

```
    }

    public void withdraw(double amount) {
        if (amount <= 0 || amount > MAX_WITHDRAWAL || amount > balance) {
            throw new IllegalArgumentException("Invalid amount: " + amount);
        }
        balance -= amount;
    }

    public int getNumber() { return number; }
    public String getOwner() { return owner; }
    public double getBalance() { return balance; }
    public static int getCreatedAccounts() { return created; }

    @Override
    public String toString() {
        return "#" + number + " " + owner + ": " + balance;
    }
}
```

2.4 A shop in two packages

```
// src/it/polito/shop/Product.java
package it.polito.shop;

public class Product {
    private final String name;
    private final double price;
    int stock;                                      // package

    Product(String name, double price, int stock) {   // package: only Shop creates products
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    public String getName() { return name; }
    public double getPrice() { return price; }
    public int getStock() { return stock; }

    void removeStock(int qty) { stock -= qty; }  // package
}

// src/it/polito/shop/Discounts.java
package it.polito.shop;

class Discounts {                                   // not public: invisible outside the package
    static final int BULK_QUANTITY = 10;
    static final double BULK_FACTOR = 0.9;

    static double apply(double price, int qty) {
        double total = price * qty;
        if (qty >= BULK_QUANTITY) {
            total = total * BULK_FACTOR;
        }
        return total;
    }
}

// src/it/polito/shop/Shop.java
```

```java
package it.polito.shop;

public class Shop {
    private final Product[] products;
    private int count = 0;

    public Shop(int capacity) {
        products = new Product[capacity];
    }

    public boolean addProduct(String name, double price, int stock) {
        if (count == products.length || find(name) != null) {
            return false;
        }
        products[count++] = new Product(name, price, stock);
        return true;
    }

    public Product find(String name) {
        for (int i = 0; i < count; i++) {
            if (products[i].getName().equals(name)) {
                return products[i];
            }
        }
        return null;
    }

    public double sell(String name, int qty) {
        Product p = find(name);
        if (p == null || qty <= 0 || qty > p.stock) {
            return -1;
        }
        p.removeStock(qty);
        return Discounts.apply(p.getPrice(), qty);
    }

    public int stockOf(String name) {
        Product p = find(name);
        return p == null ? 0 : p.getStock();
    }
}

// src/it/polito/client/ShopApp.java
package it.polito.client;

import it.polito.shop.*;        // brings in the public classes only: Shop and Product

public class ShopApp {
    public static void main(String[] args) {
        // the main of the exercise
    }
}
```

The three forbidden lines fail to compile: `p.removeStock(1)` with "removeStock(int) is not public in Product; cannot be accessed from outside package"; `new Product("x", 1, 1)` with "Product(String,double,int) is not public in Product; ..."; `Discounts.apply(1.5, 10)` with "cannot find symbol", because `import it.polito.shop.*` brings in only the public classes (with the full name, `it.polito.shop.Discounts.apply(1.5, 10)`, the error is "Discounts is not public in it.polito.shop; cannot be accessed from outside package").

2.5 Number tools

```java
import java.util.Scanner;

public class NumberTools {

    public static int sumInts(String line) {
        Scanner s = new Scanner(line);
        int sum = 0;
        while (s.hasNext()) {
            if (s.hasNextInt()) {
                sum += s.nextInt();
            } else {
                s.next();               // skip the token that is not an int
            }
        }
        return sum;
    }

    public static String stats(String line) {
        Scanner s = new Scanner(line);
        int count = 0, sum = 0;
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
        while (s.hasNext()) {
            if (s.hasNextInt()) {
                int v = s.nextInt();
                count++;
                sum += v;
                min = Math.min(min, v);
                max = Math.max(max, v);
            } else {
                s.next();
            }
        }
        if (count == 0) {
            return "count=0";
        }
        double avg = (double) sum / count;      // cast first, otherwise integer division
        return "count=" + count + " min=" + min + " max=" + max + " avg=" + avg;
    }

    public static String bases(int n) {
        return n + " = 0b" + Integer.toBinaryString(n) + " = 0x" + Integer.toHexString(n)
                + " = 0o" + Integer.toOctalString(n);
    }

    public static int percent(int part, int total) {
        return (int) Math.round(100.0 * part / total);   // Math.round(double) returns a long
    }

    public static boolean sameValue(Integer a, Integer b) {
        return a.equals(b);     // == compares objects: true only inside the cache -128..127
    }
}
```

2.6 Array tools

```java
import java.util.Arrays;

public class ArrayTools {

    public static int[][] pascal(int n) {
        int[][] t = new int[n][];
        for (int i = 0; i < n; i++) {
            t[i] = new int[i + 1];
            t[i][0] = 1;
            t[i][i] = 1;
            for (int j = 1; j < i; j++) {
                t[i][j] = t[i - 1][j - 1] + t[i - 1][j];
            }
        }
        return t;
    }

    public static int[][] transpose(int[][] m) {
        int[][] t = new int[m[0].length][m.length];
        for (int i = 0; i < m.length; i++) {
            for (int j = 0; j < m[i].length; j++) {
                t[j][i] = m[i][j];
            }
        }
        return t;
    }

    // O(1): rows are separate array objects, swapping the references is enough
    public static void swapRows(double[][] m, int i, int j) {
        double[] tmp = m[i];
        m[i] = m[j];
        m[j] = tmp;
    }

    public static double average(double... values) {
        double sum = 0;
        for (double v : values) {
            sum += v;
        }
        return sum / values.length;          // no arguments: 0.0 / 0 = NaN
    }

    public static int max(int first, int... others) {
        int res = first;
        for (int v : others) {
            if (v > res) res = v;
        }
        return res;
    }

    public static int[] concatSorted(int[] a, int[] b) {
        int[] r = Arrays.copyOf(a, a.length + b.length);
        for (int i = 0; i < b.length; i++) {
            r[a.length + i] = b[i];
        }
        Arrays.sort(r);
        return r;
    }
}
```

2.7 static, final, factory methods, singleton

```
public class Temperature {
    public static final double ABSOLUTE_ZERO = -273.15;
    private static final Temperature FREEZING = new Temperature(0.0);   // pre-built, shared

    private final double celsius;

    private Temperature(double celsius) { this.celsius = celsius; }

    public static Temperature ofCelsius(double c) {
        if (c < ABSOLUTE_ZERO) {
            throw new IllegalArgumentException("Below absolute zero: " + c);
        }
        if (c == 0.0) {
            return FREEZING;            // safe to share only because the class is immutable
        }
        return new Temperature(c);
    }

    public static Temperature ofFahrenheit(double f) {
        return ofCelsius((f - 32) * 5 / 9);
    }

    public double celsius() { return celsius; }
    public double fahrenheit() { return celsius * 9 / 5 + 32; }

    public Temperature plus(double delta) {   // "modifies" by returning a new object
        return ofCelsius(celsius + delta);
    }

    @Override
    public String toString() { return celsius + " C"; }
}

public class IdGenerator {
    private static final String PREFIX;
    private static IdGenerator instance;

    static {                                  // runs once, when the class is initialized
        PREFIX = "ID-";
        IO.println("IdGenerator class loaded");
    }

    private int next = 1;

    private IdGenerator() {
        IO.println("IdGenerator created");
    }

    public static IdGenerator getInstance() {
        if (instance == null) {
            instance = new IdGenerator();     // created at the first request only
        }
        return instance;
    }

    public String nextId() {
        return PREFIX + String.format("%04d", next++);
    }
}
```

```java
import static java.lang.Math.*;

public class Geometry {
    private Geometry() { }                    // utility class: no instances

    public static double distance(double x1, double y1, double x2, double y2) {
        return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
    }

    public static double circleArea(double r) {
        return PI * r * r;
    }
}
```

`IdGenerator class loaded` appears after `start` because a class is initialized – its static blocks run – at its first active use, here the call to `getInstance()`.

2.8 Cards: enum and record

```java
public enum Suit { SPADES, HEARTS, DIAMONDS, CLUBS }

import java.util.Arrays;

public record Card(int rank, Suit suit) {

    public String label() {
        String r;
        switch (rank) {
            case 1:  r = "A"; break;
            case 11: r = "J"; break;
            case 12: r = "Q"; break;
            case 13: r = "K"; break;
            default: r = String.valueOf(rank);
        }
        return r + " of " + suit;
    }

    public Card withRank(int newRank) {       // records are immutable: return a new Card
        return new Card(newRank, suit);
    }

    public static Card[] fullDeck() {
        Card[] deck = new Card[Suit.values().length * 13];
        int k = 0;
        for (Suit s : Suit.values()) {
            for (int r = 1; r <= 13; r++) {
                deck[k++] = new Card(r, s);
            }
        }
        return deck;
    }

    public static Card[] deal(Card[] deck, int n) {
        return Arrays.copyOfRange(deck, 0, n);
    }

    public static int[] countBySuit(Card... cards) {
        int[] counts = new int[Suit.values().length];
        for (Card c : cards) {
            counts[c.suit().ordinal()]++;
```

```
        }
        return counts;
    }

    public static Card highest(Card first, Card... others) {
        Card best = first;
        for (Card c : others) {
            if (c.rank() > best.rank()) {
                best = c;
            }
        }
        return best;
    }
}
```

The record generates the constructor, `rank()`, `suit()`, `equals`, `hashCode` and `toString` (`Card[rank=12, suit=HEARTS]`).

2.9 Nested classes

```
public class StackOfInt {

    private static class Element {    // static: needs no StackOfInt object; private: hidden detail
        int value;
        Element next;

        Element(int value, Element next) {
            this.value = value;
            this.next = next;
        }
    }

    private Element head;
    private int size;

    public void push(int v) {
        head = new Element(v, head);
        size++;
    }

    public int pop() {
        if (head == null) {
            throw new IllegalStateException("empty stack");
        }
        int v = head.value;
        head = head.next;
        size--;
        return v;
    }

    public int peek() {
        if (head == null) {
            throw new IllegalStateException("empty stack");
        }
        return head.value;
    }

    public boolean isEmpty() { return head == null; }
    public int size() { return size; }
```

```
    @Override
    public String toString() {
        String s = "[";
        for (Element e = head; e != null; e = e.next) {
            s += e.value;
            if (e.next != null) {
                s += ", ";
            }
        }
        return s + "]";
    }
}

public class Counter {
    int i;

    public class Incrementer {          // inner: each one belongs to the Counter that built it
        private int step = 1;

        public void doIncrement() {
            i += step;                   // the i of the enclosing Counter
        }

        Incrementer(int step) {
            this.step = step;
        }
    }

    public Incrementer buildIncrementer(int step) {
        return new Incrementer(step);
    }

    public int getValue() {
        return i;
    }
}
```

The local class, inside `main`, right after `int factor = 3;`:

```
class Multiplier {
    int apply(int x) {
        return x * factor;     // allowed because factor is (effectively) final
    }
}
```

Part 3 – Characters and strings

3.1 Text utilities

```
public class TextUtils {

    public static String stats(String s) {
        int letters = 0, digits = 0, spaces = 0, others = 0;
        for (char c : s.toCharArray()) {
            if (Character.isLetter(c)) {
                letters++;
            } else if (Character.isDigit(c)) {
                digits++;
```

```
            } else if (Character.isSpaceChar(c)) {
                spaces++;
            } else {
                others++;
            }
        }
        return "letters=" + letters + " digits=" + digits + " spaces=" + spaces + " others=" +
        ↪  others;
    }

    // non-overlapping occurrences; word must not be empty
    public static int countOccurrences(String text, String word) {
        int count = 0;
        int pos = text.indexOf(word);
        while (pos != -1) {
            count++;
            pos = text.indexOf(word, pos + word.length());   // pos + 1 would count overlaps
        }
        return count;
    }

    public static String capitalizeWords(String s) {
        String result = "";
        boolean startOfWord = true;
        for (char c : s.toCharArray()) {
            if (Character.isSpaceChar(c)) {
                result += c;
                startOfWord = true;
            } else if (startOfWord) {
                result += Character.toUpperCase(c);
                startOfWord = false;
            } else {
                result += Character.toLowerCase(c);
            }
        }
        return result;
    }

    public static String initials(String fullName) {
        String result = "";
        String rest = fullName.trim();
        while (!rest.isEmpty()) {
            result += Character.toUpperCase(rest.charAt(0)) + ".";
            int space = rest.indexOf(' ');
            if (space == -1) {
                break;
            }
            rest = rest.substring(space + 1).trim();
        }
        return result;
    }

    public static String extension(String fileName) {
        int dot = fileName.lastIndexOf(".");
        return dot == -1 ? "" : fileName.substring(dot + 1);
    }

    public static String firstAlphabetical(String first, String... others) {
        String best = first;
        for (String w : others) {
            if (w.compareToIgnoreCase(best) < 0) {
                best = w;
            }
```

```
        }
        return best;
    }
}
```

Strings are immutable: no method changes a string, the "modifying" ones return another string (some return the same one when nothing changes), which is why the results are built up in `result`. The slides write `subString`, but the method is `substring`.

3.2 A formatted receipt

```
public record Item(int code, String name, int qty, double unitPrice) {
    public double total() {
        return qty * unitPrice;
    }
}

import java.util.Locale;

public class Receipt {

    private static final Locale L = Locale.US;   // decimal point on any PC

    public static String header() {
        return "%-5s %-12s%4s%9s".formatted("CODE", "ITEM", "QTY", "TOTAL");   // no numbers
    }

    public static String line(Item it) {
        return String.format(L, "%05d %-12s%4d%9.2f", it.code(), it.name(), it.qty(), it.total());
    }

    public static String receipt(double discount, Item... items) {
        String r = header() + "\n";
        double sum = 0;
        for (Item it : items) {
            r += line(it) + "\n";
            sum += it.total();
        }
        r += String.format(L, "%-5s %-12s%4s%(9.2f\n", "", "Discount", "", -discount);   // ( flag
        r += String.format(L, "%-5s %-12s%4s%9.2f\n", "", "TOTAL", "", sum - discount);
        return r;
    }

    public static String footer(String cashier, int number) {
        return String.format(L, "Receipt #%2$d, served by %1$s", cashier, number);
    }
}
```

`%-12s` left-aligns in 12 characters, `%4d` right-aligns in 4, `%05d` pads with zeros, `%9.2f` gives 2 decimals in 9 characters, `%(9.2f` prints a negative number in parentheses. Without `Locale.US`, an Italian PC prints `2,40`.

3.3 StringBuilder, speed and the string pool

```java
public class BuilderTools {

    public static boolean isPalindrome(String s) {
        StringBuilder letters = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (Character.isLetter(c)) {
                letters.append(Character.toLowerCase(c));
            }
        }
        String forward = letters.toString();             // take it BEFORE reverse()...
        String backward = letters.reverse().toString();  // ...because reverse() changes the
        ↪ builder
        return forward.equals(backward);
    }

    public static String reverseWords(String s) {
        StringBuilder result = new StringBuilder();
        String rest = s.trim();
        while (!rest.isEmpty()) {
            int space = rest.lastIndexOf(' ');
            result.append(rest.substring(space + 1));     // last word (space == -1: whole string)
            if (space == -1) {
                break;
            }
            result.append(' ');
            rest = rest.substring(0, space).trim();
        }
        return result.toString();
    }

    public static String compress(String s) {
        StringBuilder sb = new StringBuilder();
        int i = 0;
        while (i < s.length()) {
            char c = s.charAt(i);
            int run = 1;
            while (i + run < s.length() && s.charAt(i + run) == c) {
                run++;
            }
            sb.append(c).append(run);                     // append returns the builder: chaining
            i += run;
        }
        return sb.toString();
    }

    public static String group(String s, int n, char sep) {
        StringBuilder sb = new StringBuilder(s);
        for (int pos = ((s.length() - 1) / n) * n; pos > 0; pos -= n) {   // right to left
            sb.insert(pos, sep);
        }
        return sb.toString();
    }
}
```

`group` inserts from the right so that the earlier positions do not move.

```
public class ConcatBenchmark {

    static String withString(int n) {
        String s = "";
        for (int i = 0; i < n; i++) {
            s += i;                    // a new String (full copy) at every step: O(n^2)
        }
        return s;
    }

    static String withBuffer(int n) {
        StringBuffer sb = new StringBuffer();      // synchronized (thread-safe)
        for (int i = 0; i < n; i++) {
            sb.append(i);
        }
        return sb.toString();
    }

    static String withBuilder(int n) {
        StringBuilder sb = new StringBuilder();    // same methods, not synchronized
        for (int i = 0; i < n; i++) {
            sb.append(i);
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        final int N = 100_000;
        long t0 = System.currentTimeMillis();
        String a = withString(N);
        long t1 = System.currentTimeMillis();
        String b = withBuffer(N);
        long t2 = System.currentTimeMillis();
        String c = withBuilder(N);
        long t3 = System.currentTimeMillis();
        IO.println("String        " + (t1 - t0) + " ms");
        IO.println("StringBuffer  " + (t2 - t1) + " ms");
        IO.println("StringBuilder " + (t3 - t2) + " ms");
        IO.println("same result: " + (a.equals(b) && b.equals(c)) + ", length " + c.length());
    }
}

public class PoolDemo {

    static int countSameObject(String[] words, String target) {
        int n = 0;
        for (String w : words) {
            if (w == target) n++;            // identity
        }
        return n;
    }

    static int countEqual(String[] words, String target) {
        int n = 0;
        for (String w : words) {
            if (w.equals(target)) n++;       // content
        }
        return n;
    }

    static void internAll(String[] words) {
        for (int i = 0; i < words.length; i++) {
            words[i] = words[i].intern();    // the pooled copy
```

```
        }
    }
}
```

Before `internAll`, only the literal `"java"` is the same object as the literal passed as target (literals are pooled); the other three are new objects with equal content. After `intern()` all four point to the pooled one.

3.4 Encoding tools

```
import java.nio.charset.Charset;

public class EncodingTools {

    static final Charset UTF8 = Charset.forName("UTF-8");
    static final Charset LATIN1 = Charset.forName("ISO-8859-1");

    public static String hex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            if (sb.length() > 0) {
                sb.append(' ');
            }
            sb.append(String.format("%02X", b & 0xFF));   // & 0xFF: bytes are signed
        }
        return sb.toString();
    }

    public static String dump(String s, String charsetName) {
        byte[] bytes = s.getBytes(Charset.forName(charsetName));
        return s + " in " + charsetName + ": " + hex(bytes) + " (" + bytes.length + " bytes)";
    }

    public static String breakIt(String s) {
        return new String(s.getBytes(UTF8), LATIN1);
    }

    // Latin-1 maps the chars 0-255 one-to-one to bytes, so the original bytes come back intact
    public static String fixMojibake(String broken) {
        return new String(broken.getBytes(LATIN1), UTF8);
    }

    public static int codePointLength(String s) {
        return s.codePointCount(0, s.length());
    }

    public static boolean canEncode(String s, String charsetName) {
        return Charset.forName(charsetName).newEncoder().canEncode(s);
    }
}
```

é is one byte (E9) in ISO-8859-1 and two (C3 A9) in UTF-8; € does not exist in ISO-8859-1, so `getBytes` replaces it with `?`. 😀 (U+1F600) is above U+FFFF, so it takes two Java `char`s (a surrogate pair) but is one code point. If the console shows `?` even with correct strings, the console encoding is wrong, not your program: hence `-Dstdout.encoding=UTF-8`.

Part 4 – JUnit 5

4.1 Test the slides' Stack

```java
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class StackTest {
    private Stack stack;

    @BeforeEach
    void setUp() {
        stack = new Stack(3);          // a fresh stack before EVERY test
    }

    @Test
    void newStackIsEmpty() {
        assertTrue(stack.isEmpty(), "A new stack should be empty");   // JUnit 5: message last
    }

    @Test
    void pushMakesStackNonEmpty() {
        assertTrue(stack.push(10));
        assertFalse(stack.isEmpty(), "Stack should not be empty after a push");
    }

    @Test
    void popReturnsElementsInLifoOrder() throws StackException {
        stack.push(1);
        stack.push(2);
        stack.push(3);
        assertEquals(3, stack.pop());
        assertEquals(2, stack.pop());
        assertEquals(1, stack.pop());
        assertTrue(stack.isEmpty());
    }

    @Test
    void pushOnFullStackReturnsFalse() throws StackException {
        assertTrue(stack.push(1));
        assertTrue(stack.push(2));
        assertTrue(stack.push(3));
        assertFalse(stack.push(4), "Push on a full stack must fail");
        assertEquals(3, stack.pop(), "The rejected element must not be stored");
    }

    @Test
    void popOnEmptyStackThrows() {
        assertThrows(StackException.class, () -> stack.pop());
    }

    @Test
    void defaultCapacityIsTen() {
        Stack s = new Stack();
        for (int i = 0; i < 10; i++) {
            assertTrue(s.push(i), "push number " + i + " should succeed");
        }
        assertFalse(s.push(10));
    }
```

```
}
```

Tests that call `pop()` outside `assertThrows` declare `throws StackException` (a checked exception); if it were thrown unexpectedly the test would end as an error.

4.2 Test the bank account, then a suite

```
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class BankAccountTest {
    private static final double EPS = 1e-9;
    private static int testsRun = 0;
    private BankAccount acc;

    @BeforeAll
    static void start() {                       // once, before all tests: must be static
        IO.println("BankAccountTest: start");
    }

    @AfterEach
    void countTest() {                          // after every test
        testsRun++;
    }

    @AfterAll
    static void end() {
        IO.println("BankAccountTest: " + testsRun + " tests run");
    }

    @BeforeEach
    void setUp() {
        acc = new BankAccount("Alice", 100);
    }

    @Test
    void initialState() {
        assertEquals("Alice", acc.getOwner());
        assertEquals(100.0, acc.getBalance(), EPS);
    }

    @Test
    void newAccountWithoutInitialBalanceIsEmpty() {
        assertEquals(0.0, new BankAccount("Bob").getBalance(), EPS);
    }

    @Test
    void depositIncreasesBalance() {
        acc.deposit(50.5);
        assertEquals(150.5, acc.getBalance(), EPS);
    }

    @Test
    void withdrawDecreasesBalance() {
        acc.withdraw(30);
```

```
        assertEquals(70.0, acc.getBalance(), EPS);
    }

    @Test
    void nonPositiveDepositIsRejectedAndBalanceUnchanged() {
        assertThrows(IllegalArgumentException.class, () -> acc.deposit(0));
        assertThrows(IllegalArgumentException.class, () -> acc.deposit(-5));
        assertEquals(100.0, acc.getBalance(), EPS);
    }

    @Test
    void cannotWithdrawMoreThanBalance() {
        assertThrows(IllegalArgumentException.class, () -> acc.withdraw(100.01));
        assertEquals(100.0, acc.getBalance(), EPS);
    }

    @Test
    void withdrawalLimitBoundary() {
        BankAccount rich = new BankAccount("Carol", 5000);
        rich.withdraw(BankAccount.MAX_WITHDRAWAL);          // exactly the limit: allowed
        assertEquals(4000.0, rich.getBalance(), EPS);
        assertThrows(IllegalArgumentException.class, () -> rich.withdraw(1000.01));
    }

    @Test
    void negativeInitialBalanceIsRejected() {
        assertThrows(IllegalArgumentException.class, () -> new BankAccount("Dan", -1));
    }

    @Test
    void numbersAreConsecutive() {
        // the counter is static, shared by every test, and the test order is not fixed:
        // assert the relation between two accounts, not absolute numbers
        BankAccount x = new BankAccount("X");
        BankAccount y = new BankAccount("Y");
        assertEquals(x.getNumber() + 1, y.getNumber());
    }
}

import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectClasses({ StackTest.class, BankAccountTest.class })   // other packages:
↪  ex4_1.StackTest.class
public class AllTests { }
```

The suite needs the `junit-platform-suite` engine (it is inside the console-standalone jar). Run from the console: `java -jar junit-platform-console-standalone.jar execute -cp <classes> --select-class AllTests`.

```java
import ex2_7.Temperature;
import it.polito.shop.Shop;

public class MiscTest {

    @Test
    void findReturnsNullForUnknownProduct() {
        Shop shop = new Shop(5);
        shop.addProduct("pen", 1.5, 20);
        assertNotNull(shop.find("pen"));
        assertNull(shop.find("eraser"));
    }

    @Test
    void freezingTemperatureIsShared() {
        assertSame(Temperature.ofCelsius(0), Temperature.ofFahrenheit(32));     // same object
        assertNotSame(Temperature.ofCelsius(20), Temperature.ofCelsius(20));    // two objects
    }

    @Test
    void neverSellsMoreThanTheStock() {
        Shop shop = new Shop(5);
        shop.addProduct("pen", 1.5, 20);
        if (shop.sell("pen", 100) != -1) {
            fail("sold more than the stock");
        }
        assertEquals(20, shop.stockOf("pen"));
    }
}
```

`assertSame`/`assertNotSame` compare with `==` (identity), `assertEquals` with `equals()`.

4.3 Bug reproduction and TDD

1. Tests, then the fix:

```
public class CapitalizeTest {
    @Test
    void lowercasesTheRest() {             // bug 2
        assertEquals("Hello", StringUtils.capitalize("hELLO"));
    }

    @Test
    void emptyStringStaysEmpty() {         // bug 1
        assertEquals("", StringUtils.capitalize(""));
    }

    @Test
    void singleLetter() {                  // already works: guards against regressions
        assertEquals("A", StringUtils.capitalize("a"));
    }

    @Test
    void nullIsRejected() {
        assertThrows(IllegalArgumentException.class, () -> StringUtils.capitalize(null));
    }
}

public class StringUtils {
    public static String capitalize(String w) {
        if (w == null) {
            throw new IllegalArgumentException("word must not be null");
        }
        if (w.isEmpty()) {
            return w;
        }
        return Character.toUpperCase(w.charAt(0)) + w.substring(1).toLowerCase();
    }
}
```

On the buggy version: `lowercasesTheRest` is a **failure** (`expected: <Hello> but was: <HELLO>`); `nullIsRejected` is a **failure** (an NPE is thrown instead of `IllegalArgumentException`); `emptyStringStaysEmpty` is an **error** (`StringIndexOutOfBoundsException: Index 0 out of bounds for length 0`, an unexpected exception); `singleLetter` passes. Eclipse and the XML report keep failures and errors apart; the JUnit 5 console prints "3 tests failed".

2. Tests first, then the code:

```
import static org.junit.jupiter.api.Assertions.*;
import static org.junit.jupiter.api.Assumptions.assumeTrue;

import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.Test;

public class RomanTest {

    @Test
    void singleSymbols() {
        assertEquals("I", Roman.toRoman(1));
        assertEquals("V", Roman.toRoman(5));
        assertEquals("X", Roman.toRoman(10));
        assertEquals("M", Roman.toRoman(1000));
```

```
    }

    @Test
    void additiveForms() {
        assertEquals("III", Roman.toRoman(3));
        assertEquals("VIII", Roman.toRoman(8));
        assertEquals("XIV", Roman.toRoman(14));
    }

    @Test
    void subtractiveForms() {
        assertEquals("IV", Roman.toRoman(4));
        assertEquals("IX", Roman.toRoman(9));
        assertEquals("XL", Roman.toRoman(40));
        assertEquals("XC", Roman.toRoman(90));
        assertEquals("CD", Roman.toRoman(400));
        assertEquals("CM", Roman.toRoman(900));
    }

    @Test
    void longNumbers() {
        assertEquals("MCMXCIV", Roman.toRoman(1994));
        assertEquals("MMXXVI", Roman.toRoman(2026));
        assertEquals("MMMCMXCIX", Roman.toRoman(3999));
    }

    @Test
    void outOfRangeIsRejected() {
        assertThrows(IllegalArgumentException.class, () -> Roman.toRoman(0));
        assertThrows(IllegalArgumentException.class, () -> Roman.toRoman(4000));
        assertThrows(IllegalArgumentException.class, () -> Roman.toRoman(-7));
    }

    @Disabled("fromRoman not implemented yet")
    @Test
    void parsesRoman() {
        assertEquals(1994, Roman.fromRoman("MCMXCIV"));
    }

    @Test
    void exhaustiveCheck() {
        assumeTrue(System.getenv("SLOW_TESTS") != null, "set SLOW_TESTS to run this test");
        for (int n = 1; n <= 3999; n++) {
            String r = Roman.toRoman(n);
            for (char c : r.toCharArray()) {
                assertTrue("MDCLXVI".indexOf(c) >= 0, "bad symbol in " + r);
            }
            assertFalse(r.contains("IIII") || r.contains("XXXX") || r.contains("CCCC"), r);
        }
    }
}

public class Roman {
    private static final int[] VALUES = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
    private static final String[] SYMBOLS = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX",
    ↪  "V", "IV", "I"};

    public static String toRoman(int n) {
        if (n < 1 || n > 3999) {
            throw new IllegalArgumentException("Out of range: " + n);
        }
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < VALUES.length; i++) {
```

```
            while (n >= VALUES[i]) {          // greedy: largest symbol first
                sb.append(SYMBOLS[i]);
                n -= VALUES[i];
            }
        }
        return sb.toString();
    }

    public static int fromRoman(String s) {
        throw new UnsupportedOperationException("not implemented yet");
    }
}
```

Putting `"CM"`, `"CD"`, `"XC"`, … in the table is what makes the subtractive tests pass with a simple greedy loop. Linux/macOS: `SLOW_TESTS=1 java -jar ...`; Windows PowerShell: `$env:SLOW_TESTS=1` first.

Part 5 – Inheritance, interfaces, lambdas

5.1 Employees, managers, CEO

```
public class Employee {
    private final String name;
    private double wage;

    public Employee(String name, double wage) {
        this.name = name;
        this.wage = wage;
    }

    public String getName() { return name; }
    public double getWage() { return wage; }
    public void incrementWage(double percent) { wage += wage * percent / 100; }
    public double totalPay() { return wage; }
    public String describe() { return name + " earns " + wage; }

    @Override
    public String toString() { return describe(); }
}

public class Manager extends Employee {
    private String unit;

    public Manager(String name, double wage, String unit) {
        super(name, wage);               // Employee has no no-argument constructor
        this.unit = unit;
    }

    public String getUnit() { return unit; }
    public void changeUnit(String newUnit) { unit = newUnit; }

    @Override
    public String describe() {
        return super.describe() + ", manages " + unit;    // reuse the parent's version
    }
}

public class CEO extends Manager {
    private final double bonus;
```

```
    public CEO(String name, double wage, double bonus) {
        super(name, wage, "the company");
        this.bonus = bonus;
    }

    @Override
    public double totalPay() { return super.totalPay() + bonus; }

    @Override
    public String describe() { return super.describe() + ", bonus " + bonus; }
}

public class Payroll {

    public static double totalPay(Employee... staff) {
        double total = 0;
        for (Employee e : staff) {
            total += e.totalPay();          // dynamic binding: the CEO adds its bonus
        }
        return total;
    }

    public static void raiseAll(double percent, Employee... staff) {
        for (Employee e : staff) {
            e.incrementWage(percent);
        }
    }

    public static int countManagers(Employee... staff) {
        int n = 0;
        for (Employee e : staff) {
            if (e instanceof Manager) {     // true for Manager AND CEO
                n++;
            }
        }
        return n;
    }

    public static Manager findManagerOf(String unit, Employee... staff) {
        for (Employee e : staff) {
            if (e instanceof Manager m && m.getUnit().equals(unit)) {   // test + cast in one step
                return m;
            }
        }
        return null;
    }
}
```

`IO.println(e)` calls `toString()`, which calls `describe()`: the version that runs is chosen at run time from the real class of the object. `staff[2]` has static type `Employee`, so calling `changeUnit` needs the downcast `(Manager)`.

5.2 equals, hashCode, toString, clone

```java
public class Student {
    private final int id;
    private final String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() { return id; }
    public String getName() { return name; }

    @Override
    public String toString() {
        return "Student[id=" + id + ", name=" + name + "]";
    }

    @Override
    public boolean equals(Object o) {          // parameter Object: an override, not an overload
        if (this == o) {
            return true;
        }
        if (!(o instanceof Student)) {
            return false;                      // also covers o == null
        }
        Student other = (Student) o;
        return this.id == other.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);           // only what equals() uses
    }
}

static int countDistinct(Object... items) {
    int distinct = 0;
    for (int i = 0; i < items.length; i++) {
        boolean seenBefore = false;
        for (int j = 0; j < i; j++) {
            if (items[i].equals(items[j])) {
                seenBefore = true;
                break;
            }
        }
        if (!seenBefore) {
            distinct++;
        }
    }
    return distinct;
}

import java.util.Arrays;

public class Polygon implements Cloneable {
    String name;
    int[] xs;

    public Polygon(String name, int... xs) {
        this.name = name;
```

```
        this.xs = xs;
    }

    @Override
    public Polygon clone() throws CloneNotSupportedException {
        Polygon copy = (Polygon) super.clone();  // field-by-field copy (allowed: we implement
        ↪  Cloneable)
        copy.xs = xs.clone();                    // without this line both objects share one array
        return copy;                             // name can be shared: String is immutable
    }

    @Override
    public String toString() {
        return name + Arrays.toString(xs);
    }
}
```

Without `implements Cloneable`, `super.clone()` throws `CloneNotSupportedException` at run time.

5.3 Abstract classes and Template Method

```
import java.util.Locale;

public abstract class Shape {
    private final String name;

    protected Shape(String name) { this.name = name; }

    public String getName() { return name; }
    public abstract double area();
    public abstract double perimeter();

    public final String describe() {           // template method, cannot be overridden
        return String.format(Locale.US, "%s: area=%.2f perimeter=%.2f", name, area(), perimeter());
    }
}

public class Circle extends Shape {
    private final double radius;

    public Circle(double radius) {
        super("Circle");
        this.radius = radius;
    }

    @Override public double area() { return Math.PI * radius * radius; }
    @Override public double perimeter() { return 2 * Math.PI * radius; }
}

public class Rectangle extends Shape {
    private final double width, height;

    public Rectangle(double width, double height) {
        this("Rectangle", width, height);
    }

    protected Rectangle(String name, double width, double height) {   // for subclasses
        super(name);
        this.width = width;
        this.height = height;
```

```
    }

    @Override public double area() { return width * height; }
    @Override public double perimeter() { return 2 * (width + height); }
}

public class Square extends Rectangle {
    public Square(double side) {
        super("Square", side, side);
    }
}

public abstract class Sorter {                  // from the slides
    public void sort(Object[] v) {
        for (int i = 1; i < v.length; ++i) {
            for (int j = 0; j < v.length - i; ++j) {
                if (compare(v[j], v[j + 1]) > 0) {
                    Object o = v[j];
                    v[j] = v[j + 1];
                    v[j + 1] = o;
                }
            }
        }
    }

    protected abstract int compare(Object a, Object b);
}

class StringSorter extends Sorter {
    @Override
    protected int compare(Object a, Object b) {
        return ((String) a).compareTo((String) b);
    }
}

class LengthSorter extends Sorter {
    @Override
    protected int compare(Object a, Object b) {
        return Integer.compare(((String) a).length(), ((String) b).length());
    }
}

class AreaSorter extends Sorter {
    @Override
    protected int compare(Object a, Object b) {
        return Double.compare(((Shape) a).area(), ((Shape) b).area());
    }
}
```

"pear" stays before "kiwi" (both 4 letters) because the bubble sort only swaps when `compare` is > 0. `new Shape("x")` or `new Sorter()` would not compile: abstract classes cannot be instantiated.

5.4 Interfaces: complex numbers, default methods

```java
import java.util.Locale;

public interface Complex {
    double real();
    double im();
    double mod();
    double arg();

    static Complex fromRect(double re, double im) { return new ComplexRect(re, im); }
    static Complex fromPolar(double mod, double arg) { return new ComplexPolar(mod, arg); }

    default boolean isReal() { return im() == 0; }

    default Complex plus(Complex other) {
        return fromRect(real() + other.real(), im() + other.im());
    }

    default Complex times(Complex other) {
        return fromPolar(mod() * other.mod(), arg() + other.arg());
    }

    default String asString() {
        return String.format(Locale.US, "%.2f%+.2fi", real(), im());
    }
}

class ComplexRect implements Complex {       // package-private: clients use the factories
    private final double re, im;
    ComplexRect(double re, double im) { this.re = re; this.im = im; }
    @Override public double real() { return re; }
    @Override public double im() { return im; }
    @Override public double mod() { return Math.sqrt(re * re + im * im); }
    @Override public double arg() { return Math.atan2(im, re); }
}

class ComplexPolar implements Complex {
    private final double mod, arg;
    ComplexPolar(double mod, double arg) { this.mod = mod; this.arg = arg; }
    @Override public double real() { return mod * Math.cos(arg); }
    @Override public double im() { return mod * Math.sin(arg); }
    @Override public double mod() { return mod; }
    @Override public double arg() { return arg; }
}

interface Walker  { default String move() { return "walk"; } }
interface Swimmer { default String move() { return "swim"; } }

interface Amphibian extends Walker, Swimmer {   // an interface may extend several interfaces
    @Override
    default String move() {                     // mandatory: the two inherited defaults clash
        return Walker.super.move() + " and " + Swimmer.super.move();
    }
}

public class Duck implements Amphibian {
}
```

Without its own `move()`, `Amphibian` does not compile (“types Walker and Swimmer are incompatible”). `X.super.move()` calls the default of a specific direct superinterface. `Duck` inherits the resolved `move()` and needs no body; the same override could also be written in a class implementing

`Walker, Swimmer` directly.

5.5 Composite expression tree

```java
public interface Expression {
    double eval();
    String formula();

    static String wrap(Expression e) {       // shared helper: parentheses only for composites
        String f = e.formula();
        if (e instanceof Composed) {
            return "(" + f + ")";
        }
        return f;
    }
}

public interface Composed { }                // flag interface: no methods

public class Value implements Expression {   // leaf
    private final double value;
    public Value(double value) { this.value = value; }
    @Override public double eval() { return value; }
    @Override public String formula() { return String.valueOf(value); }
}

public class Operation implements Expression, Composed {
    private final char op;
    private final Expression left, right;

    public Operation(char op, Expression left, Expression right) {
        if ("+-*/".indexOf(op) < 0) {
            throw new IllegalArgumentException("Unknown operator: " + op);
        }
        this.op = op;
        this.left = left;
        this.right = right;
    }

    @Override
    public double eval() {
        double l = left.eval();
        double r = right.eval();
        switch (op) {
            case '+': return l + r;
            case '-': return l - r;
            case '*': return l * r;
            case '/': return l / r;
            default: throw new IllegalStateException("unreachable");
        }
    }

    @Override
    public String formula() {
        return Expression.wrap(left) + op + Expression.wrap(right);
    }
}

public class Negate implements Expression, Composed {
    private final Expression operand;
    public Negate(Expression operand) { this.operand = operand; }
```

```
    @Override public double eval() { return -operand.eval(); }
    @Override public String formula() { return "-" + Expression.wrap(operand); }
}
```

5.6 Comparable, Comparator, Strategy

```
public class Student implements Comparable<Student> {
    private final int id;
    private final String name;
    private final double average;

    public Student(int id, String name, double average) {
        this.id = id;
        this.name = name;
        this.average = average;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getAverage() { return average; }

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.id, other.id);    // this.id - other.id can overflow
    }

    @Override
    public String toString() { return id + ":" + name + "(" + average + ")"; }
}

// the slides' StudentCmp, fixed: generic (no casts), so a and b are already Students (the slide
↪  used a.id on two Objects), and no en dash
class ByIdDescending implements Comparator<Student> {
    @Override
    public int compare(Student a, Student b) {
        return Integer.compare(b.getId(), a.getId());
    }
}

// in main, after (1) and (2):
Arrays.sort(sv, new Comparator<Student>() {             // 3 anonymous class: by name
    @Override
    public int compare(Student a, Student b) {
        return a.getName().compareTo(b.getName());
    }
});
IO.println(Arrays.toString(sv));

Arrays.sort(sv, (a, b) -> {                              // 4 lambda: average desc, then name
    int c = Double.compare(b.getAverage(), a.getAverage());
    return c != 0 ? c : a.getName().compareTo(b.getName());
});
IO.println(Arrays.toString(sv));

Arrays.sort(sv, Comparator.comparing(Student::getName).reversed());   // 5 method reference
IO.println(Arrays.toString(sv));
```

`Comparable` gives the class one natural order; each `Comparator` is a strategy object chosen by the caller.

5.7 Iterable and Iterator

```java
import java.util.Iterator;
import java.util.NoSuchElementException;

public class Range implements Iterable<Integer> {
    private final int from, to, step;

    public Range(int from, int to, int step) {
        if (step <= 0) {
            throw new IllegalArgumentException("step must be positive");
        }
        this.from = from;
        this.to = to;
        this.step = step;
    }

    @Override
    public Iterator<Integer> iterator() {
        return new RangeIterator();          // a new, independent cursor every time
    }

    private class RangeIterator implements Iterator<Integer> {   // inner: reads from, to, step
        private int current = from;

        @Override
        public boolean hasNext() { return current < to; }

        @Override
        public Integer next() {
            if (!hasNext()) {
                throw new NoSuchElementException();
            }
            int value = current;
            current += step;
            return value;                    // autoboxing int -> Integer
        }
    }
}
```

`AnonRange` is identical except for `iterator()`:

```
@Override
public Iterator<Integer> iterator() {
    return new Iterator<Integer>() {
        private int current = from;

        @Override
        public boolean hasNext() { return current < to; }

        @Override
        public Integer next() {
            if (!hasNext()) {
                throw new NoSuchElementException();
            }
            int value = current;
            current += step;
            return value;
        }
    };
}
```

The explicit loop that a for-each turns into:

```
Iterator<Integer> it = new Range(0, 10, 3).iterator();
int sum = 0;
while (it.hasNext()) {
    int i = it.next();
    sum += i;
}
IO.println(sum);
```

5.8 Functional interfaces, lambdas, method references

```
static void forEachIf(int[] v, IntPredicate cond, IntConsumer action) {
    for (int x : v) {
        if (cond.test(x)) {
            action.accept(x);
        }
    }
}

static int reduce(int[] v, int identity, IntBinaryOperator op) {
    int acc = identity;
    for (int x : v) {
        acc = op.applyAsInt(acc, x);
    }
    return acc;
}

static int[] generate(int n, IntSupplier s) {
    int[] r = new int[n];
    for (int i = 0; i < n; i++) {
        r[i] = s.getAsInt();
    }
    return r;
}

static String[] mapToStrings(int[] v, IntFunction<String> f) {
    String[] r = new String[v.length];
```

```
    for (int i = 0; i < v.length; i++) {
        r[i] = f.apply(v[i]);
    }
    return r;
}

// 3. the same Consumer three ways
Consumer<Object> p1 = new Consumer<Object>() {
    @Override
    public void accept(Object o) {
        IO.println(o);
    }
};
Consumer<Object> p2 = o -> IO.println(o);
Consumer<Object> p3 = IO::println;
p1.accept("A");
p2.accept("B");
p3.accept("C");
```

The lambdas of part 2 replaced by method references (same output):

```
IO.println(reduce(v, 0, Integer::sum));
IO.println(reduce(v, Integer.MIN_VALUE, Math::max));
IO.println(Arrays.toString(mapToStrings(new int[]{10, 255}, Integer::toHexString)));
IntFunction<Character> hex = hexDigits::charAt;
Function<String, Integer> len = String::length;
Supplier<StringBuilder> maker = StringBuilder::new;
BiFunction<String, String, Boolean> same = String::equalsIgnoreCase;
Predicate<String> empty = String::isEmpty;
Consumer<Object> printer = IO::println;
```

Kinds of method reference used: `Integer::sum`, `Math::max`, `Integer::toHexString`, `IO::println` are static; `hexDigits::charAt` is an instance method of a given object; `String::length`, `String::equalsIgnoreCase`, `String::isEmpty` are instance methods of a type (the first argument is the target); `StringBuilder::new` is a constructor. A lambda can only read local variables that are effectively final, which is why the counter captured by the lambda passed to `generate` is a one-element array.

5.9 Observer by composition

```
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

public class Thermostat {
    private final PropertyChangeSupport pcs = new PropertyChangeSupport(this);   // composition
    private double temperature;

    public Thermostat(double initial) { temperature = initial; }

    public double getTemperature() { return temperature; }

    public void setTemperature(double newValue) {
        double old = temperature;
        temperature = newValue;
        pcs.firePropertyChange("temperature", old, newValue);   // no event if old equals new
    }

    public void addListener(PropertyChangeListener l) {         // delegation
        pcs.addPropertyChangeListener("temperature", l);
```

```
    }

    public void removeListener(PropertyChangeListener l) {
        pcs.removePropertyChangeListener("temperature", l);
    }
}

import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;

public class Display implements PropertyChangeListener {
    @Override
    public void propertyChange(PropertyChangeEvent evt) {
        IO.println("Display: " + evt.getOldValue() + " -> " + evt.getNewValue());
    }
}

// the alarm: PropertyChangeListener has one method, so a lambda works
t.addListener(evt -> {
    double v = (Double) evt.getNewValue();
    if (v > 30) {
        IO.println("ALARM: " + v);
    }
});

public class ThermostatTest {
    private Thermostat t;
    private int events;
    private double lastValue;
    private PropertyChangeListener recorder;

    @BeforeEach
    void setUp() {
        t = new Thermostat(20.0);
        events = 0;
        recorder = evt -> {            // a lambda may modify fields (not local variables)
            events++;
            lastValue = (Double) evt.getNewValue();
        };
        t.addListener(recorder);
    }

    @Test
    void changeIsNotified() {
        t.setTemperature(25.0);
        assertEquals(1, events);
        assertEquals(25.0, lastValue, 1e-9);
        assertEquals(25.0, t.getTemperature(), 1e-9);
    }

    @Test
    void sameValueIsNotNotified() {
        t.setTemperature(25.0);
        t.setTemperature(25.0);
        assertEquals(1, events, "setting the same value twice must notify once");
    }

    @Test
    void removedListenerIsNotNotified() {
        t.removeListener(recorder);
        t.setTemperature(30.0);
        assertEquals(0, events);
        assertEquals(30.0, t.getTemperature(), 1e-9);
```

```
    }
}
```

Composition keeps `Thermostat` free to extend another class; the slides' alternative, `extends Observable`, uses up the single superclass and `Observable` is deprecated since Java 9.

Slide errata

These slide lines do not compile as written, or are wrong or outdated in Java 25; each code row was checked with `javac` 25 (snippets in `slide-errata/`). Code copied from the PDFs also brings typographic quotes and dashes (’, “, –), which are illegal characters in Java source.

Deck	The slide says	Correct Java
J02	`Counter` chain calls `decrement(7)`	the method is never defined (see 2.2)
J02	`String in = java.lang.IO.readLine();`	`IO.readln()`
J02	`public static int inverse(double n){ return 1 / n; }`	return type `double` (“possible lossy conversion from double to int”)
J02	`Car.countCountBuiltCars`	`Car.countBuiltCars`
J02	`public final static double 2PI;`	an identifier cannot start with a digit, and Math.acos(-1) is π, not 2π: put 2 * Math.acos(-1) in a field named `TWO_PI`
J02	record vs class: `class Point(){ ... }`	`class Point { ... }`
J02	`StackOfInt`: `private Element head` and `public int void pop()`	`private Element head;` and `public int pop()`
J02	an inner class “cannot be instantiated from a static method / other classes”	both work given an outer object, even inside a static method: `c.new Incrementer(2)` (if the constructor is visible)
J02, J04	`new Integer(88)`, `new Character('b')`, `finalize()`	still compile but deprecated (`finalize` is marked for removal): use `Integer.valueOf` / autoboxing
J02a	`"Human".subString(2)`	`substring`
J02a	‘è’ has code point `U+00C8`	`U+00E8` (U+00C8 is ‘È’); its UTF-8 bytes C3 A8 are right
J02a	UTF-16 uses 16 bits for `U+0`–`U+D7FF`	also for `U+E000`–`U+FFFF`; only code points above U+FFFF take 32 bits
J03	`Stack.pop()`: `throw new StackException()`	missing `;`, and `StackException` must be declared
J03	`@BeforeClass` / `@AfterClass`	JUnit 4 names; JUnit 5 uses static `@BeforeAll` / `@AfterAll`
J03	`@Test(expected=PossibleException.class)`	JUnit 4 only; JUnit 5 uses `assertThrows`
J03	`assumeTrue("X not available", res.isAvailable())`	JUnit 5: `assumeTrue(res.isAvailable(), "X not available")`
J03	suite with `@RunWith(JUnitPlatform.class)` / `@RunWith(Suite.class)`	JUnit 5: `@Suite` + `@SelectClasses` (org.junit.platform.suite.api)
J04	`super(...)` must be the first statement	Java 25 (JEP 513) allows statements before `super(...)` if they do not use the object being built, e.g. argument checks; calling an instance method there is still an error
J04	`Operation`'s constructor is named `Addition`	`public Operation(char o, Expression l, Expression r)`

Deck	The slide says	Correct Java
J04	`class Value implements Expression { @Override void value(){…} @Override void formula(){…} }`	`public double eval()` and `public String formula()`
J04	`Objects[3]= 42;`	`objects[3] = 42;`
J04	static methods in interfaces "can be overridden"	they are not inherited, so they cannot be overridden: "static methods cannot be annotated with `@Override`"
J04	comparators `return a.id – b.id;`	`Integer.compare(sa.id, sb.id)` (see 5.6)
J04	`printer.consume("Hello!")`	`Consumer`'s method is `accept`
J02a	`Charset` methods `getDecoder()` / `getEncoder()`	`newDecoder()` / `newEncoder()` (used in 3.4)
J01	`jar cvfm my.jar manifest.txt`	adds no classes: `jar cvfm my.jar manifest.txt *.class` (and the manifest must end with a newline)
J02	`final static float PI = 3.14;`	`3.14` is a `double`: `3.14f`, or declare `PI` as `double` ("possible lossy conversion from double to float")
J02	`Character toUpper()`, `toLower()`	`toUpperCase()`, `toLowerCase()`
J02	`Integer.valueof(i)`	`Integer.valueOf(i)`
J02	`Incrementer byOne = c.buildIncrementer(1);` written outside `Counter`	`Counter.Incrementer byOne = ...` (as in 2.9); also `Counter c = new Counter()` needs its `;`
J02	records "since Java 17"	standard since Java 16 (JEP 395)
J02a	`boolean equals(String s)`	the real signature is `equals(Object o)` – see 5.2
J03	`assertEquals(1.0, Math.cos(3.14), 0.01)`	fails: `cos(3.14)` is about -1, so the expected value is `-1.0`
J03	JUnit 5 assertions include `assertThat()` for Hamcrest	Jupiter's `Assertions` has no `assertThat`: use Hamcrest's `org.hamcrest.MatcherAssert.assertThat`
J04	`float f; f = 4.7;  // legal`	illegal (`double` to `float`): `f = 4.7f;`
J04	`class Employee{ String toString(){…} }`	`public String toString()`: an override cannot reduce visibility
J04	abstract `Sorter`: an extra `}` before `abstract int compare(...)`	remove that `}`; if `compare` is made `protected` (as in 5.3), the subclasses' `compare` must be `protected` or `public`
J04	`ComplexPolar`: `mod*cos(arg)`, `mod*sin(arg)`	`Math.cos` / `Math.sin`, or `import static java.lang.Math.*`
J04	`Iterator it = seq.iterator()` … `IO.println(el);`	missing `;` and the variable is `element`, not `el`
J04	Observer `Subject`: `String prop="ini";` … `property = val;`	`prop = val;`
J04	`java.lang.Comparator`, `java.utils.Arrays`	`java.util.Comparator`, `java.util.Arrays`